

Лабораторная работа № 4

Эмуляция и измерение задержек в глобальных сетях

Шияпова Дарина Илдаровна

Содержание

1	Цель работы	4
2	Задание	5
3	Теоретическое введение	6
4	Выполнение лабораторной работы	7
4.1	Добавление/изменение задержки в эмулируемой глобальной сети	9
4.2	Изменение задержки в эмулируемой глобальной сети	12
4.3	Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети	13
4.4	Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети	15
4.5	Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети	17
4.6	Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети	18
4.7	Воспроизведение экспериментов. Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети	18
5	Выводы	24
	Список литературы	25

Список иллюстраций

4.1	Исправление прав запуска X-соединения	7
4.2	ifconfig на хостах h1 и h2	8
4.3	Проверка подключения между хостами	9
4.4	Добавление задержки в 100мс	11
4.5	Двунаправленная задержка соединения	12
4.6	Изменение задержки на 50мс	13
4.7	Восстановление исходных значений задержки	14
4.8	Добавление значения дрожания задержки в интерфейс подключения	16
4.9	Добавление значения корреляции для джиттера и задержки в ин- терфейс подключения	17
4.10	Распределение задержки в интерфейсе подключения	18
4.11	скрипт для визуализации ping_plot	21
4.12	Makefile для управления процессом проведения эксперимента . .	21
4.13	результате выполнения скриптов	22
4.14	результате выполнения скриптова	22
4.15	Скрипт rtt.py	23
4.16	Результат работы скрипта rtt.py	23

1 Цель работы

Основной целью работы является знакомство с NETEM — инструментом для тестирования производительности приложений в виртуальной сети, а также получение навыков проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.

2 Задание

1. Задайте простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8.
2. Проведите интерактивные эксперименты по добавлению/изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети.
3. Реализуйте воспроизводимый эксперимент по заданию значения задержки в эмулируемой глобальной сети. Постройте график.
4. Самостоятельно реализуйте воспроизводимые эксперименты по изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети. Постройте графики.

3 Теоретическое введение

Mininet[**mininet?**] – это эмулятор компьютерной сети. Под компьютерной сетью подразумеваются простые компьютеры — хосты, коммутаторы, а так же OpenFlow-контроллеры. С помощью простейшего синтаксиса в примитивном интерпретаторе команд можно разворачивать сети из произвольного количества хостов, коммутаторов в различных топологиях и все это в рамках одной виртуальной машины(ВМ). На всех хостах можно изменять сетевую конфигурацию, пользоваться стандартными утилитами(`ifconfig`, `ping`) и даже получать доступ к терминалу. На коммутаторы можно добавлять различные правила и маршрутизировать трафик.

4 Выполнение лабораторной работы

Запустим виртуальную среду с mininet. Из основной ОС подключимся к виртуальной машине. В виртуальной машине mininet при необходимости исправим права запуска X-соединения. Скопируем значение куки (MIT magic cookie) своего пользователя mininet в файл для пользователя root (рис. 4.1).

```
mininet@mininet-vm:~$ xauth list $DISPLAY
mininet-vm/unix:11 MIT-MAGIC-COOKIE-1 f0cb9ac988b1dac1b6396527ccb5060
mininet@mininet-vm:~$ sudo -i
root@mininet-vm:~# xauth add mininet-vm/unix:11 MIT-MAGIC-COOKIE-1 f0cb9ac988b1dac1b6396527ccb5060
root@mininet-vm:~# logout
mininet@mininet-vm:~$
```

Рис. 4.1: Исправление прав запуска X-соединения

Задайте простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8 (рис. ??).

После введения этой команды запустятся терминалы двух хостов, коммутатора и контроллера. Терминалы коммутатора и контроллера можно закрыть.

Простейшая топология

На хостах h1 и h2 введем команду `ifconfig`, чтобы отобразить информацию, относящуюся к их сетевым интерфейсам и назначенным им IP-адресам. В дальнейшем при работе с NETEM и командой `tc` будут использоваться интерфейсы `h1-eth0` и `h2-eth0` (рис. 4.2).

```
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Running terms on localhost:11.0
*** Starting controller
c0 Xl1 connection rejected because of wrong authentication.

c11 connection rejected because of wrong authentication.
*** Starting 1 switches
s1 ...Xl1 connection rejected because of wrong authentication.
c11 connection rejected because of wrong authentication.

*** Starting CLI:
```

Рис. 4.2: ifconfig на хостах h1 и h2

Проверим подключение между хостами h1 и h2 с помощью команды ping с параметром -c 6 (рис. 4.3).


```
root@mininet-vn:/home/mininet# ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether ee:cf:43:e2:c4:1a txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1646 bytes 359400 (359.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1646 bytes 359400 (359.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vn:/home/mininet#

root@mininet-vn:/home/mininet# ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
    ether ae:8b:31:4f:fe:28 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2198 bytes 389792 (389.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2198 bytes 389792 (389.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vn:/home/mininet#
```

Рис. 4.3: Проверка подключения между хостами

4.1 Добавление/изменение задержки в эмулируемой глобальной сети

На хосте h1 добавим задержку в 100 мс к выходному интерфейсу (рис. 4.4).

```
sudo tc qdisc add dev h1-eth0 root netem delay 100ms
```

- sudo: выполнить команду с более высокими привилегиями;
- tc: вызвать управление трафиком Linux;

- `qdisc`: изменить дисциплину очередей сетевого планировщика;
- `add`: создать новое правило;
- `dev h1-eth0`: указать интерфейс, на котором будет применяться правило;
- `netem`: использовать эмулятор сети;
- `delay 100ms`: задержка ввода 100 мс.

Проверим, что соединение от хоста `h1` к хосту `h2` имеет задержку 100 мс, используя команду `ping` с параметром `-c 6` с хоста `h1`

```
root@mininet-vn:/home/mininet# ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1646 bytes 359400 (359.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1646 bytes 359400 (359.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vn:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.207 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.056 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.047 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5086ms
rtt min/avg/max/mdev = 0.047/1.861/10.741/3.971 ms
root@mininet-vn:/home/mininet#

root@mininet-vn:/home/mininet# ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
    ether ae:8b:31:4f:fe:28 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2198 bytes 389792 (389.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2198 bytes 389792 (389.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vn:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.51 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.048 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.050 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.042 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.045 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.048 ms

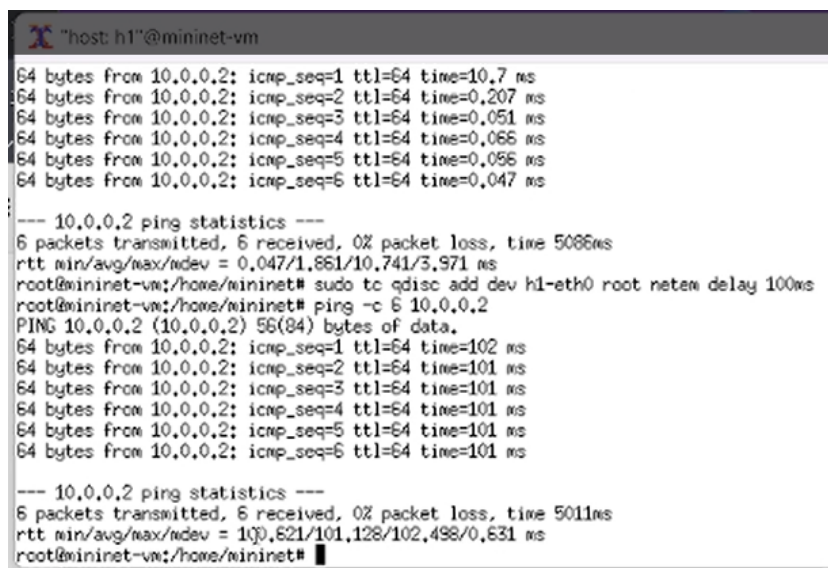
--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5088ms
rtt min/avg/max/mdev = 0.042/0.456/2.505/0.916 ms
root@mininet-vn:/home/mininet#
```

Рис. 4.4: Добавление задержки в 100мс

Для эмуляции глобальной сети с двунаправленной задержкой необходимо к соответствующему интерфейсу на хосте h2 также добавим задержку в 100 миллисекунд (рис. 4.5).

Проверим, что соединение между хостом h1 и хостом h2 имеет RTT в 200 мс

(100 мс от хоста h1 к хосту h2 и 100 мс от хоста h2 к хосту h1), повторив команду ping с параметром -c 6 на терминале хоста h1.



```
host: h1@mininet-vm
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.207 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.086 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.047 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5086ms
rtt min/avg/max/mdev = 0.047/1.861/10.741/3.971 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5011ms
rtt min/avg/max/mdev = 100.621/101.128/102.498/0.631 ms
root@mininet-vm:/home/mininet#
```

Рис. 4.5: Двухнаправленная задержка соединения

4.2 Изменение задержки в эмулируемой глобальной сети

Изменим задержку со 100 мс до 50 мс для отправителя h1 и для получателя h2 (рис. 4.6).

Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс, используя команду ping с параметром -c 6 с терминала хоста h1.

```
root@h1:~# ping 10.0.0.2
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.207 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.047 ms
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 508ms
rtt min/avg/max/mdev = 0.047/1.861/10.741/3.971 ms
root@mininet-vx:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms
root@mininet-vx:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 501ms
rtt min/avg/max/mdev = 100.821/101.128/102.498/0.631 ms
root@mininet-vx:/home/mininet# sudo tc qdisc change dev h1-eth0 root netem delay 100ms
```

Рис. 4.6: Изменение задержки на 50мс

4.3 Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети

Восстановим конфигурацию по умолчанию, удалив все правила, применённые к сетевому планировщику соответствующего интерфейса для отправителя h1 и для получателя h2. Проверим, что соединение между хостом h1 и хостом h2 не имеет явно установленной задержки, используя команду ping с параметром -c 6 с терминала хоста h1 (рис. 4.7).

```
host: h1" @mininet-vm
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms
:
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5011ms
rtt min/avg/max/ndev = 100.621/101.128/102.498/0.631 ms
root@mininet-vm:/home/mininet# sudo tc qdisc change dev h1-eth0 root netem delay 50
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5013ms
rtt min/avg/max/ndev = 100.687/101.868/103.474/1.002 ms
root@mininet-vm:/home/mininet#

host: h2" @mininet-vm

root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.51 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.048 ns
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.050 ns
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.042 ns
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.045 ns
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.048 ns
--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5088ms
rtt min/avg/max/ndev = 0.042/0.456/2.505/0.916 ns
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem delay 100ns
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=203 ns
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=201 ns
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=201 ns
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=201 ns
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=201 ns
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=201 ns
--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5011ms
rtt min/avg/max/ndev = 200.647/201.308/202.957/0.760 ns
root@mininet-vm:/home/mininet# sudo tc qdisc change dev h2-eth0 root netem delay 50n
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=102 ms
```

Рис. 4.7: Восстановление исходных значений задержки

4.4 Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети

Добавим на узле h1 задержку в 100 мс со случайным отклонением 10 мс. Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс со случайным отклонением ± 10 мс, используя в терминале хоста h1 команду `ping` с параметром -с 6. Восстановим конфигурацию интерфейса по умолчанию на узле h1 (рис. 4.8).


```

X "host: h1" @mininet-vm

64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5013ms
rtt min/avg/max/ndev = 100.687/101.688/103.474/1.002 ms
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.50 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.043 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.120 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.044 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.049 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.075 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5079ms
rtt min/avg/max/ndev = 0.043/0.471/2.500/0.907 ms
root@mininet-vm:/home/mininet#

X "host: h2" @mininet-vm

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5011ns
rtt min/avg/max/ndev = 200.647/201.308/202.957/0.760 ns
root@mininet-vm:/home/mininet# sudo tc qdisc change dev h2-eth0 root netem delay 50ns
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=102 ns
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=101 ns
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=102 ns
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=102 ns
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=102 ns
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=101 ns

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5011ns
rtt min/avg/max/ndev = 100.829/101.735/102.531/0.496 ns
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h2-eth0 root netem
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=3.72 ns
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.493 ns
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.172 ns
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.049 ns
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.061 ns
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.051 ns

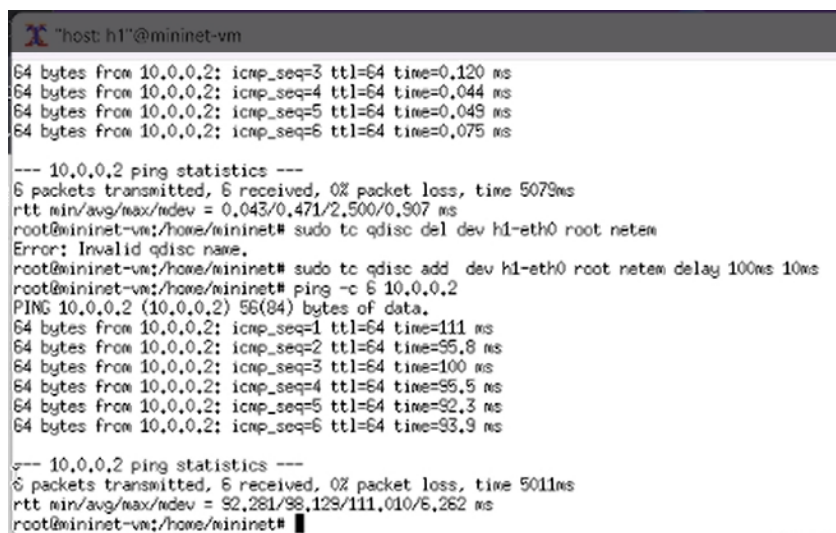
--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5077ns
rtt min/avg/max/ndev = 0.049/0.757/3.721/1.334 ns
root@mininet-vm:/home/mininet#

```

Рис. 4.8: Добавление значения дрожания задержки в интерфейс подключения

4.5 Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети

Добавим на интерфейсе хоста h1 задержку в 100 мс с вариацией ± 10 мс и значением корреляции в 25%. Убедимся, что все пакеты, покидающие устройство h1 на интерфейсе h1-eth0, будут иметь время задержки 100 мс со случайным отклонением ± 10 мс, при этом время передачи следующего пакета зависит от предыдущего значения на 25%. Используем для этого в терминале хоста h1 команду ping с параметром -c 20. Восстановим конфигурацию интерфейса по умолчанию на узле h1 (рис. 4.9).



```
host: h1@mininet-vn
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.120 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.044 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.049 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.075 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5079ms
rtt min/avg/max/mdev = 0.043/0.471/2.500/0.907 ms
root@mininet-vn:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
Error: Invalid qdisc name.
root@mininet-vn:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms
root@mininet-vn:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=111 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=95.8 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=95.5 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=92.3 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=93.9 ms

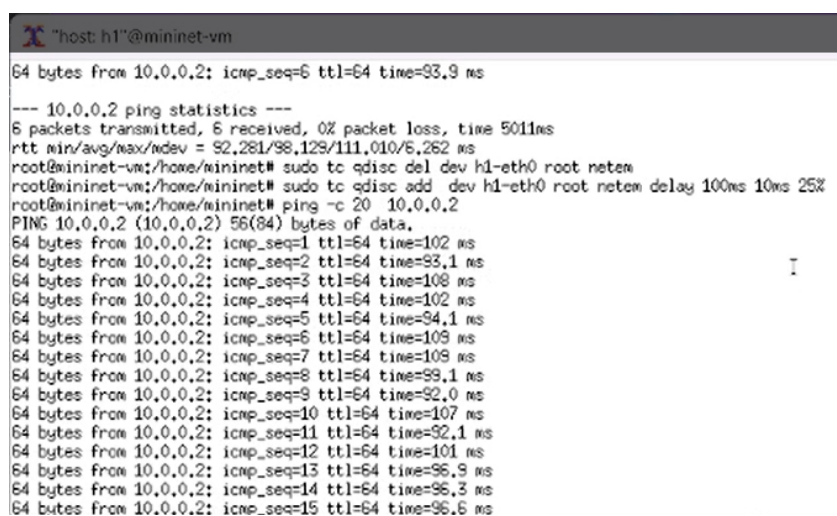
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5011ms
rtt min/avg/max/mdev = 92.281/98.129/111.010/6.262 ms
root@mininet-vn:/home/mininet#
```

Рис. 4.9: Добавление значения корреляции для джиттера и задержки в интерфейс подключения

4.6 Распределение задержки в интерфейсе

подключения к эмулируемой глобальной сети

Зададим нормальное распределение задержки на узле h1 в эмулируемой сети. Убедимся, что все пакеты, покидающие хост h1 на интерфейсе h1-eth0, будут иметь время задержки, которое распределено в диапазоне 100 мс $\pm$ 20 мс. Используем для этого команду ping на терминале хоста h1 с параметром -c 10. Восстановим конфигурацию интерфейса по умолчанию на узле h1. Завершим работу mininet в интерактивном режиме (рис. 4.10).



```
host: h1@mininet-vm
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=93.9 ms
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 501ms
rtt min/avg/max/mdev = 92.281/98.129/111.010/6.262 ms
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms 25%
root@mininet-vm:/home/mininet# ping -c 20 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=93.1 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=108 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=94.1 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=99.1 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=92.0 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=107 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=92.1 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=96.9 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=96.3 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=96.6 ms
```

Рис. 4.10: Распределение задержки в интерфейсе подключения

4.7 Воспроизведение экспериментов. Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети

С помощью API Mininet воспроизведем эксперимент по добавлению задержки для интерфейса хоста, подключающегося к эмулируемой глобальной сети. В виртуальной среде mininet в своём рабочем каталоге с проектами создадим каталог simple-delay и перейдем в него.

Создадим скрипт для эксперимента lab_netem_i.py:

```
1 #!/usr/bin/env python
2
3 """
4 Simple experiment.
5 Output: ping.dat
6 """
7
8 from mininet.net import Mininet
9 from mininet.node import Controller
10 from mininet.cli import CLI
11 from mininet.log import setLogLevel, info
12 import time
13
14 def emptyNet():
15
16 "Create an empty network and add nodes to it."
17
18 net = Mininet( controller=Controller,
↵ waitConnected=True )
19
20 info( '*** Adding controller\n' )
21 net.addController( 'c0' )
22
23 info( '*** Adding hosts\n' )
24 h1 = net.addHost( 'h1', ip='10.0.0.1' )
25 h2 = net.addHost( 'h2', ip='10.0.0.2' )
26
27 info( '*** Adding switch\n' )
```

```

28 s1 = net.addSwitch( 's1' )
29
30 info( '*** Creating links\n' )
31 net.addLink( h1, s1 )
32 net.addLink( h2, s1 )
33
34 info( '*** Starting network\n' )
35 net.start()
36
37 info( '*** Set delay\n' )
38 h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem delay 100ms' )
39 h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem delay 100ms' )
40
41 time.sleep(10) # Wait 10 seconds
42
43 info( '*** Ping\n' )
44 h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time=" | awk \'{print $5, $7}\'' | sed
45
46 info( '*** Stopping network' )
47 net.stop()
48
49 if __name__ == '__main__':
50 setLogLevel( 'info' )
51 emptyNet()

```

Создаём скрипт для визуализации ping_plot результатов эксперимента (рис. 4.11).

```
root@mininet:~# ping 10.0.0.2
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=92.1 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19038ms
rtt min/avg/max/mdev = 32.013/100.263/109.438/6.054 ms
root@mininet:~# sudo tc qdisc del dev hl-eth0 root netem
root@mininet:~# sudo tc qdisc add dev hl-eth0 root netem delay 100ms 20ms distribution normal
root@mininet:~# ping -c 10 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=92.0 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=120 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=144 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=120 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=95.8 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=111 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=83.2 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=99.0 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=63.8 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=151 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9003ms
rtt min/avg/max/mdev = 63.773/106.945/150.938/26.090 ms
root@mininet:~#
```

Рис. 4.11: скрипт для визуализации ping_plot

Зададим права доступа к файлу скрипта: `chmod +x ping_plot`.

Создадим Makefile для управления процессом проведения эксперимента (рис. 4.12).

```
mininet@mininet-vm: ~/work/lab_netem_i/simple-
GNU nano 4.8
#!/usr/bin/gnuplot --persist

set terminal png crop
set output 'ping.png'
set xlabel "Sequence number"
set ylabel "Delay (ms)"
set grid
plot "ping.dat" with lines
```

Рис. 4.12: Makefile для управления процессом проведения эксперимента

Выполним эксперимент. Продemonстрируем построенный в результате выполнения скриптов график (рис. 4.13).

```
mininet@mininet-vm: ~/work/lab_netem_i/simple_delay
GNU nano 4.8 Makefile
all: ping.dat ping.png

ping.dat:
    sudo python lab_netem_i.py
    sudo chown mininet:mininet ping.dat

ping.png: ping.dat
    ./ping_plot

clean:
    rm -f *.dat *.png
```

Рис. 4.13: результате выполнения скриптов

Из файла ping.dat удалим первую строку и заново построим график. Продемонстрируем построенный в результате график (рис. 4.14).

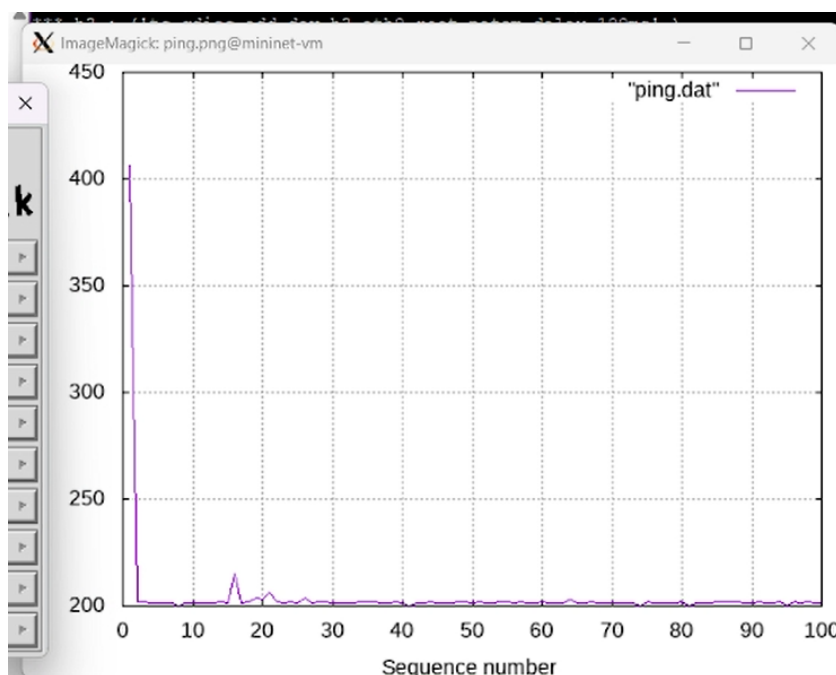


Рис. 4.14: результате выполнения скрипта

Разработайте скрипт для вычисления на основе данных файла ping.dat минимального, среднего, максимального и стандартного отклонения времени приёма-передачи (рис. 4.15).

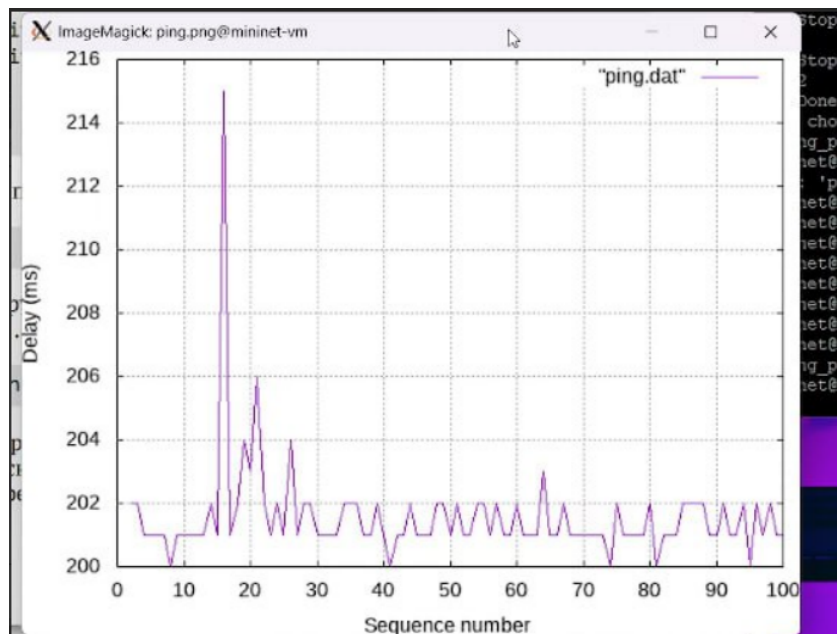


Рис. 4.15: Скрипт rtt.py

Продemonстрируем работу скрипта с выводом значений на экран или в отдельный файл (рис. 4.16).

```
mininet@mininet-vm: ~/work/lab_netem_i/simple-delay
GNU nano 4.8      rtt.py      Modified
with open('ping.dat', 'r') as f:
    s = []
    for line in f.readlines():
        if '\n' in line:
            line.replace('\n', '')
            s.append([int(j) for j in (line.split(" "))])
    s = [j[1] for j in s]
    std = (sum([(i-(sum(s)/len(s)))**2 for i in s])/(len(s)-1))**0.5
    print(f"min: {min(s)} \max: {max(s)} \n avg: {sum(s)/len(s)} \n std: {std}")
```

Рис. 4.16: Результат работы скрипта rtt.py

5 Выводы

В результате выполнения данной лабораторной работы я познакомилась с NETEM – инструментом для тестирования производительности приложений в виртуальной сети, а также получила навыки проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.

Список литературы